K-Nearest Neighbours Classifier

K-Nearest Neighbours (KNN) is a **supervised machine learning algorithm** used for:

- Classification
- Regression

It is a **distance-based**, **instance-based (lazy learning)** algorithm.

No training phase in traditional sense.
Model stores training data and makes predictions based on nearest neighbors.

Core Idea

For a new data point:

1. Compute distance from all training points
2. Select K nearest points
3. For classification → majority voting
4. For regression → average of neighbors

Important Characteristics

- Non-parametric
- Lazy learning (no explicit model training)
- Sensitive to scaling
- Works well for small to medium datasets
- Simple and interpretable

Distance Metrics

1) Euclidean Distance

Distance = √Σ(x_i - y_i)^2

2) Manhattan Distance

Distance = Σ |x_i - y_i|

3) Minkowski Distance

Distance = (Σ |x_i - y_i|^p)^(1/p)

Special cases:

- p = 1 → Manhattan
- p = 2 → Euclidean

Classification Types

- Binary classification
- Multiclass classification

Case Study – Ball Classification

Encoding Rules

- Tennis = 1
- Cricket = 2
- Rough = 1
- Smooth = 0

Dataset

Features: [Weight, Surface]

Weight	Surface	Ball
35	1	Tennis
40	1	Tennis
55	1	Tennis
65	1	Tennis
70	1	Tennis
92	0	Cricket
95	0	Cricket
100	0	Cricket
105	0	Cricket
110	0	Cricket

Internal Working – Step by Step

Step 1 – Choose K

Example: K = 3

Step 2 – Calculate Distance

New ball: [85, 0]

Compute Euclidean distance:

Example for [92,0]:

Distance = √((85-92)^2 + (0-0)^2)

Distance = √(49)
Distance = 7

Compute for all training points.

Step 3 – Sort Distances

Select 3 nearest neighbors.

Step 4 – Majority Voting

If nearest 3 are:

Cricket, Cricket, Cricket → Prediction = Cricket

Choosing K Value

- Small K → Low bias, High variance
- Large K → High bias, Low variance

Common choice:
K = √N

For 10 samples:
K ≈ 3

Important Parameters of KNeighborsClassifier

```
KNeighborsClassifier(
 n_neighbors=5,
 weights='uniform',
 algorithm='auto',
 leaf_size=30,
 p=2,
 metric='minkowski'
)
```

n_neighbors

Number of neighbors (K)
Default = 5

weights

- 'uniform' → equal voting
- 'distance' → closer points get higher weight

Distance weighting formula:
Weight = 1 / Distance

algorithm

- 'auto'
- 'ball_tree'
- 'kd_tree'
- 'brute'

Used internally for faster neighbor search.

leaf_size

Used in tree-based search structures.
Impacts speed, not accuracy.

p

Power parameter for Minkowski metric.

- p=1 → Manhattan
- p=2 → Euclidean

metric

Distance metric.

Default = 'minkowski'

Scaling Requirement

KNN is distance-based.

Features must be scaled.

Use:

- StandardScaler
- MinMaxScaler

Without scaling, feature with larger numeric range dominates.

Complete Python Code

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

# Features: [Weight, Surface]
X = [
 [35,1], [40,1], [55,1], [65,1], [70,1],
 [92,0], [95,0], [100,0], [105,0], [110,0]
]

# Labels: 1=Tennis, 2=Cricket
y = [1,1,1,1,1, 2,2,2,2,2]

# Scaling (important for KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)

# Predict new ball
new_ball = [[85,0]]
new_ball_scaled = scaler.transform(new_ball)

prediction = model.predict(new_ball_scaled)[0]

print("Prediction:", "Tennis" if prediction == 1 else "Cricket”
```

Example 2 :

```
data = [
 {'point': 'A', 'x': 1, 'y': 2, 'label': 'Red'},
 {'point': 'B', 'x': 2, 'y': 3, 'label': 'Red'},
 {'point': 'C', 'x': 3, 'y': 1, 'label': 'Blue'},
 {'point': 'D', 'x': 6, 'y': 5, 'label': 'Blue'}
]
```

Points:

Point	x	y	Label
A	1	2	Red
B	2	3	Red
C	3	1	Blue
D	6	5	Blue

Distance Formula (Euclidean)

Single-line formula:

Distance = √((x1 - x2)^2 + (y1 - y2)^2)

Example: Classify New Point P(2,2)

Assume:

New Point = (2, 2)

Step 1 – Compute Distance to All Points

Distance to A(1,2)

√((2-1)^2 + (2-2)^2)
= √(1^2 + 0)
= √1
= 1

Distance to B(2,3)

√((2-2)^2 + (2-3)^2)
= √(0 + 1)
= 1

Distance to C(3,1)

√((2-3)^2 + (2-1)^2)
= √(1 + 1)
= √2
≈ 1.41

Distance to D(6,5)

√((2-6)^2 + (2-5)^2)
= √(16 + 9)
= √25
= 5

Step 2 – Sort by Distance

Point	Distance	Label
A		1 Red
B		1 Red
C		1 Blue
D		5 Blue

Case 1 – K = 1

Nearest neighbor → A (Red)

Prediction = Red

Case 2 – K = 3

Nearest 3 neighbors:

A (Red)
B (Red)
C (Blue)

Majority = Red (2 vs 1)

Prediction = Red

Complete Python Code

```
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Dataset
X = np.array([
 [1,2],
 [2,3],
 [3,1],
 [6,5]
])

y = np.array(['Red', 'Red', 'Blue', 'Blue'])

# Create model
model = KNeighborsClassifier(n_neighbors=3)

# Fit model
model.fit(X, y)

# New point
new_point = np.array([[2,2]])

prediction = model.predict(new_point)

print("Predicted Label:", prediction[0])
```